

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



CSC10014 - TƯ DUY TÍNH TOÁN
HKII: 2025 - 2026

HỆ THỐNG LƯU TRỮ DU LỊCH THÔNG MINH

Lớp : CQ2024/6
Nhóm thực hiện : Nhóm 12
Giáo viên hướng dẫn : Hồ Tuấn Thanh
GVHDTH : Mai Anh Tuấn
Trợ giảng : Phạm Nguyễn Sơn Tùng

Thành Viên:

24120033: Đào Tiến Đạt.
24120040: Dương Hoài Đức.
24120304: Trần Thị Bích Hạnh.
24120310: Nguyễn Đình Hiếu.
24120483: Hoàng Văn Trường.

THÀNH PHỐ HỒ CHÍ MINH, NGÀY 18/6/2026

Lời Cảm Ơn

Trong quá trình thực hiện đồ án môn học, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ và hướng dẫn quý báu từ các thầy cô và bạn bè.

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến thầy **Hồ Tuấn Thanh** – giảng viên phụ trách học phần, đã cung cấp những kiến thức nền tảng cũng như tạo điều kiện để chúng em có cơ hội tiếp cận với các công nghệ và quy trình phát triển ứng dụng thực tế. Chúng em cũng xin chân thành cảm ơn thầy **Mai Anh Tuấn** – giảng viên hướng dẫn thực hành cùng các thầy cô trợ giảng đã tận tình hỗ trợ, giải đáp các thắc mắc trong quá trình học tập và thực hiện đồ án.

Thông qua học phần này, các thành viên trong nhóm không chỉ tích lũy được những kiến thức chuyên môn quý báu mà còn rèn luyện được tư duy giải quyết vấn đề, kỹ năng làm việc nhóm và khả năng vận dụng kiến thức vào các bài toán thực tế. Những kinh nghiệm thu được trong quá trình thực hiện đề tài sẽ là nền tảng quan trọng cho việc học tập và nghiên cứu trong tương lai.

Mặc dù đã nỗ lực hoàn thành đồ án với tinh thần học hỏi và trách nhiệm cao nhất, song do kiến thức và kinh nghiệm thực tiễn còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ Thầy/Cô để có thể tiếp tục hoàn thiện kiến thức và kỹ năng của mình.

Cuối cùng, chúng em xin kính chúc quý Thầy/Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công trong sự nghiệp giảng dạy và nghiên cứu.

Trân trọng,
Các thành viên Nhóm 12

Mục lục

1	Group Member	4
2	Idea	5
2.1	Đặt vấn đề	5
2.2	Tầm nhìn ứng dụng	5
2.3	Người dùng mục tiêu	5
3	Problem Analysis	5
3.1	Hành trình Người dùng	5
3.2	Phân rã Bài toán Tính toán	6
3.3	Ràng buộc, Giả định và Tiêu chí	6
4	System Overview	7
4.1	Tính Năng Cốt Lõi	7
4.2	Mình họa Tính năng	7
4.3	Điểm Dột Phá	9
5	Pattern Recognition	10
6	Abstraction	12
7	System/Algorithm Design	13
7.1	Biểu diễn Bài toán	13
7.2	Tổng quan Công nghệ	13
7.3	Mô hình C4	14
7.3.1	Cấp 1 — System Context	14
7.3.2	Cấp 2 — Container	14
7.3.3	Cấp 3 — Component	15
7.3.4	Cấp 4 — Code (Chi tiết Module AI)	16
7.4	Thiết kế Giải thuật theo Chức năng	16
7.4.1	Điều phối AI	16
7.4.2	Tìm kiếm Lai (Hybrid Search)	17
7.4.3	Gợi ý Xếp hạng	18
7.4.4	Đường ống RAG	19
7.4.5	Các Giải thuật Hỗ trợ	19
7.5	Đánh giá Hiệu năng và Khả năng Mở rộng	19
8	Implementation	19
8.1	Từ Mô hình Thiết kế đến Phân rã Module	19
8.2	Di trú Kiến trúc: Microservices → Modular Monolith	20
8.3	Các Thách thức và Giải pháp	20
8.3.1	Tính phi tất định của LLM	20
8.3.2	Streaming SSE qua POST	21
8.3.3	Tích hợp Đa Nhà cung cấp Bản đồ	21
8.3.4	Vector Embedding và Prisma	21
8.3.5	Xác thực Firebase	21

8.4	Các Thách thức Khác	22
8.5	Kết luận	22
9	Testing	22
9.1	Chiến lược Kiểm thử	22
9.2	Phạm vi Kiểm thử	22
9.2.1	Frontend — Giao diện và Tương tác	22
9.2.2	Backend — Xử lý Nghiệp vụ và Tích hợp	23
9.2.3	Các Tình huống và Giả lập Đặc thù	23
9.3	Kết quả Kiểm thử	24
9.4	Cải tiến Sau Kiểm thử	25
9.5	Hướng Phát triển	25
10	Demo	26
11	Deployment	26
12	Logbook: Plan and Actual Workload	26
13	AI usage and declaration	26
14	Kết luận và hướng phát triển	26
14.1	Kết luận	26
14.2	Những kiến thức và kỹ năng đạt được	26
14.3	Hướng phát triển trong tương lai	27

1. Group Member

STT	MSSV	Họ và Tên	Vai trò / Nhiệm vụ chính
1	24120033	Đào Tiến Đạt	
2	24120040	Dương Hoài Đức	
3	24120304	Trần Thị Bích Hạnh	
4	24120310	Nguyễn Đình Hiếu	
5	24120483	Hoàng Văn Trường	

2. Idea

2.1. Đặt vấn đề

Hiện nay, việc tìm kiếm một nơi lưu trú phù hợp khi đi du lịch không hề đơn giản. Người dùng thường phải cân nhắc nhiều yếu tố như giá cả, vị trí, tiện nghi, chất lượng dịch vụ và các đánh giá từ những khách hàng trước đó. Quá trình này vẫn còn gặp nhiều khó khăn:

- ✖ **Thông tin phân tán:** Dữ liệu về khách sạn, homestay rải rác trên nhiều nền tảng, khiến người dùng mất nhiều thời gian so sánh.
- ✖ **Bộ lọc cứng nhắc:** Các công cụ truyền thống chỉ cho phép lọc theo bộ lọc có sẵn, không hỗ trợ tìm kiếm dựa trên nhu cầu tự nhiên và cá nhân hóa của người dùng.

💡 Giải pháp đề xuất

Nhóm xây dựng hệ thống tìm kiếm và gợi ý lưu trú thông minh, cho phép người dùng mô tả nhu cầu bằng **ngôn ngữ tự nhiên**. Hệ thống sẽ phân tích yêu cầu, tìm kiếm các địa điểm phù hợp, đánh giá, xếp hạng và đưa ra những gợi ý trực quan, giúp người dùng ra quyết định nhanh chóng.

2.2. Tầm nhìn ứng dụng

Dự án không chỉ nhằm xây dựng một nền tảng thương mại thông thường, mà định hình lại cách người dùng tương tác với dữ liệu. Thông qua việc ứng dụng AI và kỹ thuật **RAG**, hệ thống kỳ vọng tạo ra một tiền đề cho tương lai, nơi AI đóng vai trò như một người điều phối đáng tin cậy.

2.3. Người dùng mục tiêu

- 👤 **Khách du lịch tự túc:** Cần tìm kiếm nhanh chóng, so sánh giá để tối ưu hóa trải nghiệm.
- 👤 **Người dùng On-site:** Đang lưu trú tại địa điểm, sẵn sàng đóng góp thông tin thực tế và trả lời câu hỏi cộng đồng.
- 👤 **Người ưu tiên cá nhân hóa:** Thích tương tác với AI thay vì tốn hàng giờ với các bộ lọc phức tạp.

3. Problem Analysis

3.1. Hành trình Người dùng

Thay vì tiếp cận theo hướng kỹ thuật khô khan, nhóm thiết kế hệ thống thông qua chuỗi câu hỏi dẫn dắt từ góc nhìn người dùng:

- 🔍 **Tìm kiếm thế nào?** Cần giao diện linh hoạt (bản đồ, lọc tiêu chí, từ khóa) kết hợp LLM để phân tích ý định thô.
- 🔗 **So sánh ra sao?** Cung cấp bảng đối chiếu tiện ích và nhận xét tổng hợp từ AI để ra quyết định khách quan.
- 🔍 **Hiểu sâu địa điểm?** AI khai thác hàng ngàn bài đánh giá, trả lời trực tiếp các câu hỏi chi tiết về wifi, bữa sáng,...
- ✅ **Chống "ảo giác" AI?** Sử dụng dữ liệu từ người dùng "On-site" để đảm bảo tính xác thực tối đa.

3.2. Phân rã Bài toán Tính toán

Từ nhu cầu thực tế, hệ thống được phân rã thành các module kỹ thuật chuyên biệt:

- ⚙️ **Intent Parsing:** LLM chuyển đổi ngôn ngữ tự nhiên thành JSON cấu trúc.
- ⚙️ **Data Aggregation:** Truy xuất đồng thời DB nội bộ và API đối tác.
- ⚙️ **Semantic Search:** Số hóa văn bản thành vector embeddings (pgvector).
- ⚙️ **Workflow Engine:** Xây dựng khung làm việc cứng (guardrails) kiểm soát AI.

3.3. Ràng buộc, Giả định và Tiêu chí

Để đảm bảo tính khả thi trong phạm vi đề án, hệ thống được thiết kế và đánh giá dựa trên các khía cạnh rõ ràng sau:

🔗 1. Ràng buộc

- ▶ **Về mặt kiến trúc:** Lựa chọn mô hình *Modular Monolith* để giảm thiểu chi phí quản lý và vận hành trong giai đoạn MVP, đồng thời vẫn giữ được tính phân tách cao giúp dễ dàng mở rộng sang Microservices sau này.
- ▶ **Về mặt tài nguyên:** Hệ thống offload (chuyển giao) toàn bộ tải tính toán AI nặng nề sang các API của bên thứ ba (như Groq). Nhờ đó, hệ thống lõi có thể chạy cục bộ mượt mà trên phần cứng cơ bản không yêu cầu GPU.
- ▶ **Về phạm vi hỗ trợ:** Ứng dụng ở giai đoạn này tập trung vào thị trường Việt Nam. Hệ thống tập trung xác định các địa điểm lưu trú trong nước, yêu cầu AI phải hỗ trợ tiếng Việt tốt và hiểu rõ ngữ cảnh văn hóa địa phương.

💡 2. Giả định

- ▶ Khả năng lập luận, phân tích ngữ nghĩa và đưa ra quyết định của Mô hình ngôn ngữ lớn là yếu tố khách quan, tính chính xác phụ thuộc vào API cung cấp chứ không bị giới hạn bởi hệ thống nội bộ.
- ▶ Dữ liệu thu thập từ các nhà cung cấp bên thứ ba (Goong, SerpAPI) luôn đảm bảo độ tin cậy về mặt nội dung và ổn định về thời gian phản hồi (uptime).

☰ 3. Tiêu chí đánh giá

- ▶ **Sự hài lòng của người dùng:** Đánh giá mức độ thoải mái, mức độ cá nhân hóa và sự hài lòng tổng thể khi người dùng trải nghiệm các tính năng tìm kiếm, trò chuyện cùng AI trên nền tảng.
- ▶ **Tính hữu dụng:** Ứng dụng phải giải quyết được triệt để bài toán tìm kiếm chỗ ở. Giao diện cần trực quan, dễ thao tác và có luồng tương tác mượt mà phù hợp với nhiều nhóm đối tượng.
- ▶ **Độ chính xác của thông tin:** Đảm bảo chất lượng dữ liệu trả về. Danh sách địa điểm phải sát thực tế, và đặc biệt câu trả lời phân tích chi tiết từ AI phải chuẩn xác, tuyệt đối không được xảy ra tình trạng ảo giác.

4. System Overview

Hệ thống là nền tảng tìm kiếm và quản lý lưu trú thông minh, kết hợp tra cứu truyền thống và sức mạnh Trí tuệ Nhân tạo. Dự án mang đến trải nghiệm đột phá qua giao diện hội thoại và tìm kiếm ngữ nghĩa.

4.1. Tính Năng Cốt Lõi

STT	Tính Năng	Mô Tả Ngắn
1	🔍 Tìm kiếm và Gợi ý	Trải nghiệm tìm kiếm đa chiều kết hợp linh hoạt giữa bộ lọc và hệ thống gợi ý cá nhân hóa.
2	🗣️ Trợ lý Ảo AI	Chatbot thông minh hỗ trợ tìm kiếm và chất lọc thông tin lưu trú qua hội thoại tự nhiên.
3	💬 Quản lý Cộng đồng	Không gian tương tác, hỏi đáp và chia sẻ nhận xét trực tiếp tại trang của từng địa điểm.
4	📍 Xác nhận On-site	Tính năng check-in độc đáo để xác nhận người dùng đang có mặt tại địa điểm thực tế.
5	📷 Đóng góp Nội dung	Trao quyền cho người dùng tải lên hình ảnh và cập nhật tình trạng mới nhất của chỗ ở.
6	❤️ Lưu trữ Địa điểm	Công cụ cá nhân để đánh dấu, lưu lại và so sánh các danh sách địa điểm yêu thích.

4.2. Minh họa Tính năng

Thay vì tiếp cận theo hướng kỹ thuật, hệ thống được thiết kế dựa trên quá trình tối ưu hóa trải nghiệm thực tế của người dùng thông qua các tính năng sau:

✎ 1. Trải nghiệm Tìm kiếm Linh hoạt

Hệ thống khắc phục hạn chế của các bộ lọc tìm kiếm truyền thống bằng cách cho phép người dùng nhập trực tiếp các yêu cầu phức tạp dưới dạng ngôn ngữ tự nhiên. Ví dụ: *"Tìm một home-stay yên tĩnh ở Đà Lạt cho gia đình 4 người, có bếp nướng BBQ và giá dưới 2 triệu."* Thông qua khả năng phân tích ngữ nghĩa, hệ thống sẽ trích xuất chính xác các tiêu chí và trả về danh sách địa điểm phù hợp nhất.

Ảnh minh họa (0.6 width)
(Tìm kiếm)

💬 2. Hỏi đáp Trực tiếp cùng Trợ lý Ảo

Ảnh minh họa (0.6 width)
(Chat AI)

Nhằm giảm thiểu thời gian đọc và tổng hợp thông tin từ hàng loạt đánh giá dài, người dùng có thể đặt câu hỏi trực tiếp cho AI tại trang thông tin của điểm lưu trú. Ví dụ: *"Khách sạn này ăn sáng có đa dạng không? Tốc độ mạng wifi có ổn định không?"*. Trợ lý ảo sẽ tự động phân tích các đánh giá thực tế từ cơ sở dữ liệu để đưa ra câu trả lời khách quan, ngắn gọn và bám sát thắc mắc của người dùng.

✓ 3. Chống Thông tin Sai lệch qua cơ chế "On-site"

Để giải quyết rủi ro sai lệch thông tin giữa quảng cáo và thực tế, hệ thống tích hợp tính năng xác thực "Check-in" dành riêng cho những khách hàng đang có mặt tại điểm lưu trú. Những hình ảnh, đánh giá hoặc câu trả lời được cung cấp bởi nhóm người dùng "On-site" này sẽ được gán nhãn xác thực và ưu tiên hiển thị, qua đó nâng cao độ tin cậy của thông tin trên nền tảng.

Ảnh minh họa (0.6 width)
(On-site)

👥 4. Tương tác Cộng đồng Trực tiếp

Ảnh minh họa (0.6 width)
(Cộng đồng)

Mỗi địa điểm trên nền tảng được tổ chức như một diễn đàn độc lập. Người dùng có thể đặt các câu hỏi đặc thù và nhận phản hồi trực tiếp từ những du khách đã hoặc đang lưu trú tại đó. Tính năng này tạo ra một vòng lặp chia sẻ thông tin liên tục, giúp dữ liệu luôn được cập nhật và phản ánh đúng tình trạng hiện tại của điểm đến.

4.3. Điểm Đột Phá

🔑 Các điểm đổi mới nổi bật

- ✓ **AI-Driven Workflow:** AI không chỉ sinh chữ. Nó là "bộ định tuyến" gọi API, tính toán logic và vẽ thẳng kết quả lên UI.
- ✓ **Semantic Search và Vector DB:** Khám phá chỗ ở thông qua độ tương đồng Cosine của ngữ nghĩa, vượt ra khỏi giới hạn so khớp từ khóa cứng nhắc.
- ✓ **On-site Insights:** Giải quyết triệt để sự sai lệch giữa quảng cáo và thực tế bằng cách ưu tiên tương tác từ người dùng thực chứng.

5. Pattern Recognition

Để thiết kế một kiến trúc hệ thống chặt chẽ, nhóm đã tiến hành phân tích và nhận diện các hành vi lặp lại của người dùng cũng như các cấu trúc tiềm ẩn trong luồng dữ liệu thực tế.

🔍 1. User Query Pattern

Quan sát thực tế cho thấy, dù người dùng có cách diễn đạt rất đa dạng khi tìm kiếm nơi lưu trú, các câu truy vấn này luôn xoay quanh một tập hợp các đặc trưng cốt lõi lặp lại. Những yếu tố này bao gồm các thông số định lượng như vị trí, ngân sách, loại hình lưu trú, tiện nghi, và các yếu tố định tính (yêu cầu mềm) như "yên tĩnh", "gần trung tâm" hay "phù hợp cho gia đình". Nhận diện được mẫu cấu trúc tiềm ẩn này, **quy tắc tổng quát** được rút ra là: mọi câu lệnh phức tạp đều có thể được bóc tách thành các tham số rời rạc kết hợp với ngữ nghĩa mở rộng. Quy tắc này trực tiếp định hình quyết định thiết kế hệ thống phân tích ý định (intent parsing). Thay vì cố gắng khớp chuỗi nguyên văn, kiến trúc hệ thống sử dụng mô hình ngôn ngữ để trích xuất và định tuyến các thông tin này vào một cấu trúc tham số chuẩn hóa, làm đầu vào cho các bước truy vấn tiếp theo.

📦 2. Accommodation Data Pattern

Trong quá trình thu thập thông tin từ nhiều nhà cung cấp khác nhau, nhóm phát hiện ra một sự tương đồng lớn về cấu trúc dữ liệu. Dù định dạng đầu vào có khác biệt, mọi điểm lưu trú đều chia sẻ các thuộc tính chung cốt lõi như tên, tọa độ địa lý, địa chỉ, mức giá, điểm đánh giá, tiện nghi, loại hình và các bài nhận xét. Mẫu cấu trúc này chỉ ra rằng, dữ liệu đa nguồn hoàn toàn có thể được ánh xạ về một khuôn dạng chung. Dựa trên quy tắc này, nhóm đã đưa ra **quyết định thiết kế** một lược đồ dữ liệu và các đối tượng truyền tải dữ liệu (DTO) thống nhất. Mọi luồng dữ liệu từ Goong, SerpAPI hay cơ sở dữ liệu nội bộ đều phải đi qua bước chuẩn hóa này, đảm bảo hệ thống có thể xử lý và hiển thị thông tin nhất quán bất kể nguồn gốc.

⚖️ 3. Decision-making Pattern

Hành vi của người dùng trong quá trình lựa chọn chỗ ở hiếm khi là một đường thẳng. Quan sát cho thấy, du khách thường không chọn ngay kết quả đầu tiên được đề xuất. Thay vào đó, họ có xu hướng lưu lại một vài lựa chọn nổi bật, đặt chúng lên bàn cân để so sánh các yếu tố như giá cả, vị trí và tiện ích trước khi đưa ra quyết định cuối cùng. Nhận ra mẫu hành vi vòng lặp này, **quy tắc tổng quát** được xác định là quá trình tìm kiếm luôn đi kèm với nhu cầu đối chiếu đa chiều. Từ đó, hệ thống được thiết kế bổ sung một luồng tính năng chuyên biệt cho việc đánh dấu và lưu trữ địa điểm. Điều này giúp trải nghiệm không bị gián đoạn và hỗ trợ người dùng ra quyết định một cách tự tin hơn.

★ 4. Review và RAG Pattern

Khi đi sâu vào các câu hỏi mà người dùng thường đặt ra về một khách sạn cụ thể, một mẫu chủ đề rõ nét xuất hiện. Hầu hết các thắc mắc đều lặp lại quanh các khía cạnh: độ sạch sẽ, chất lượng mạng kết nối, mức độ tiếng ồn, vị trí thực tế so với quảng cáo và thái độ phục vụ. Từ sự hội tụ về chủ đề này, **quy tắc rút ra** là phần lớn giá trị của các bài đánh giá nằm ở việc trả lời các câu hỏi cụ thể này. Nhận thức được điều đó, thay vì để người dùng đọc toàn bộ văn bản dài, nhóm **quyết định thiết kế** tổ chức dữ liệu đánh giá thành các đoạn thông tin nhỏ (chunks) và sử dụng kỹ thuật tìm kiếm vector. Thiết kế này cho phép hệ thống tự động trích xuất đúng phần nhận xét chứa câu trả lời và tổng hợp lại cho người dùng.

⚠ 5. Failure Pattern

Hoạt động của các dịch vụ tích hợp bên ngoài luôn tiềm ẩn những rủi ro có tính quy luật. Quá trình kiểm thử cho thấy các lỗi thường lặp lại theo những dạng nhất định: thiếu thông tin về giá, không có điểm đánh giá, dữ liệu trả về bị trùng lặp, lỗi kết nối hoặc hoàn toàn không tìm thấy kết quả phù hợp. Mẫu lỗi này dẫn đến **quy tắc**: sự thiếu hụt hoặc sai lệch dữ liệu ngoại vi là điều tất yếu và hệ thống lỗi phải có khả năng tự phục hồi. Áp dụng quy tắc này vào **quyết định thiết kế**, các cơ chế xử lý lỗi biên, khử trùng lặp (deduplication) và kịch bản dự phòng (fallback) đã được xây dựng. Khi một nhà cung cấp gặp sự cố hoặc dữ liệu không toàn vẹn, hệ thống sẽ tự động bù đắp bằng các luồng thông tin khác hoặc cấu trúc lại giao diện phù hợp mà không làm sập toàn bộ ứng dụng.

Tóm lại, hệ thống không đơn thuần là một tập hợp các tính năng rời rạc được ghép vá lại với nhau. Thay vào đó, toàn bộ kiến trúc được định hình vững chắc dựa trên quá trình quan sát, khái quát hóa và vận dụng các **mẫu lặp lại (patterns)** đặc thù của bài toán lưu trữ. Điều này đảm bảo nền tảng hoạt động nhất quán, dễ dàng mở rộng và thực sự giải quyết đúng những vấn đề cốt lõi của người dùng.

Mẫu được nhận diện

Quy tắc tổng quát

Quyết định thiết kế

User Query Pattern
Đa dạng cách diễn đạt nhưng chung các tiêu chí cốt lõi.

Các yêu cầu phức tạp có thể bóc tách thành tham số và ngữ nghĩa.

Xây dựng luồng phân tích ý định để trích xuất tham số có cấu trúc.

Data Pattern
Điểm lưu trữ đa nguồn đều chung các thuộc tính cốt lõi.

Dữ liệu đa nguồn có thể được ánh xạ về một khuôn dạng chuẩn duy nhất.

Thiết kế DTO và lược đồ dữ liệu thống nhất làm đầu mỗi chuẩn hóa.

Decision-making
Người dùng cân nhắc nhiều lựa chọn trước khi chốt đặt phòng.

Hành trình tìm kiếm luôn đi kèm nhu cầu đối chiếu đa chiều.

Tích hợp tính năng lưu trữ và theo dõi địa điểm song song tìm kiếm.

Review/RAG Pattern
Thắc mắc lặp lại các chủ đề (sạch sẽ, wifi, tiếng ồn).

Giá trị của đánh giá nằm ở khả năng giải đáp các thắc mắc cụ thể này.

Chia nhỏ đánh giá thành vector để truy xuất tự động qua RAG.

Failure Pattern
Thiếu giá, trùng lặp, lỗi dịch vụ bên thứ ba.

Lỗi ngoại vi là tất yếu, hệ thống lỗi cần tự bảo vệ và phục hồi.

Xây dựng cơ chế fallback, khử trùng lặp và các kịch bản kiểm thử biên.

Bảng 1: Pattern Recognition Summary

6. Abstraction

7. System/Algorithm Design

7.1. Biểu diễn Bài toán

Khái niệm: Biểu diễn bài toán là quá trình chuyển đổi bài toán thực tế (xây dựng nền tảng khám phá chỗ ở thông minh) thành mô hình, cấu trúc dữ liệu và ký hiệu mà máy tính có thể xử lý.

Cách nhóm áp dụng 5 dạng biểu diễn:

1. **Biểu diễn dữ liệu:** Các thực thể thực tế (người dùng, địa điểm, đánh giá, hội thoại, tệp tin) được ánh xạ thành 13 mô hình quan hệ trong PostgreSQL. Mỗi thực thể được gán kiểu dữ liệu tương ứng: UUID cho định danh, JSONB cho dữ liệu linh hoạt, vector cho nhúng ngữ nghĩa.
2. **Biểu diễn cấu trúc:** Mỗi quan hệ giữa người dùng—địa điểm—nội dung được mô hình hóa qua ERD với khóa ngoại và chỉ mục. Sự phụ thuộc giữa các module được biểu diễn bằng sơ đồ lớp và C4 Model.
3. **Biểu diễn quy trình:** Luồng tương tác (tìm kiếm → gợi ý → trò chuyện AI → check-in) được mô hình hóa qua sơ đồ tuần tự và flowchart.
4. **Biểu diễn toán học:** Công thức tính điểm gợi ý đa yếu tố (rating, ngân sách, khoảng cách, tiện ích, cận kề) với trọng số và hàm chuẩn hóa.
5. **Biểu diễn logic:** Quy tắc định tuyến ý định người dùng (IF-THEN) qua LLM JSON Mode; logic kiểm tra điều kiện trong workflow registry.

Ba nguyên tắc cốt lõi được tuân thủ:

- **Trừu tượng hóa:** Chỉ lưu nội bộ địa điểm người dùng tương tác, ủy thác khám phá cho API ngoài — loại bỏ chi tiết không cần thiết.
- **Mô hình hóa có cấu trúc:** Mỗi module có ranh giới rõ ràng (controller/service/DTO), dữ liệu và quan hệ được đồng bộ qua Prisma schema.
- **Đa biểu diễn:** Cùng một bài toán (tìm kiếm chỗ ở) được biểu diễn đồng thời dưới dạng flowchart (luồng), công thức (điểm số) và mã giả (giải thuật hợp nhất).

7.2. Tổng quan Công nghệ

Hệ thống sử dụng ngăn xếp công nghệ thống nhất:

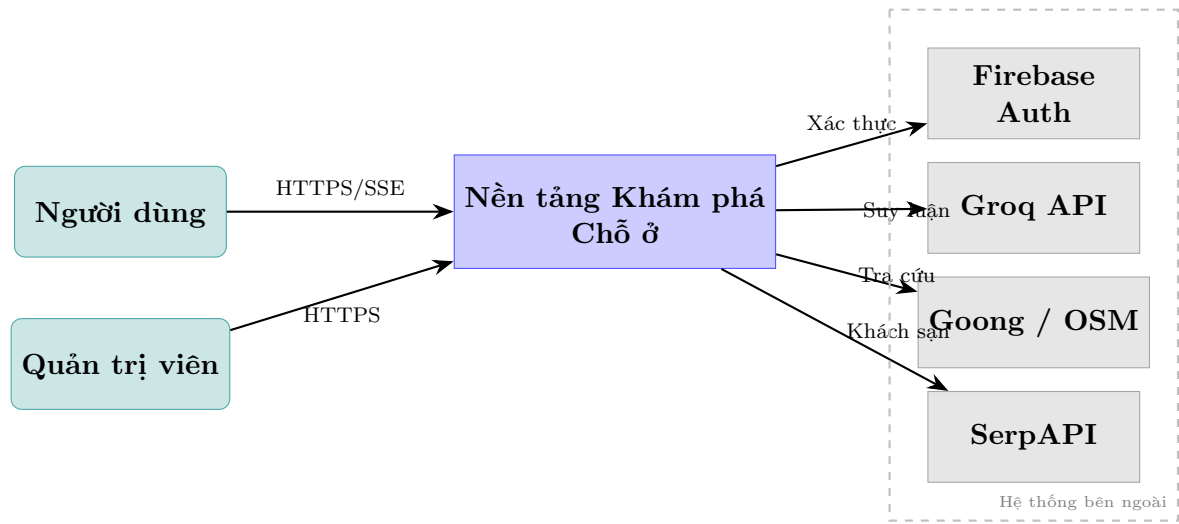
Tầng	Công nghệ	Mục đích
Giao diện	React 18 + Vite 7 + TailwindCSS 3.4	SPA, bản đồ Leaflet, responsive
Xác thực	Firebase Auth 10 (Web) + Admin SDK 12	Đăng nhập, JWT token
Backend	NestJS 10 + TypeScript 5	API REST, SSE streaming
ORM	Prisma 7.6	Truy xuất dữ liệu, migrations
Cơ sở dữ liệu	PostgreSQL 16 + pgvector	Dữ liệu quan hệ + vector nhúng
AI	Groq API (llama-3.1-70b)	Định tuyến ý định, tổng hợp phản hồi
Hạ tầng	Docker Compose V2 + Nginx 1.25	Triển khai, reverse proxy

Tích hợp bên ngoài: Goong API (tìm kiếm địa điểm chính), OpenStreetMap/Nominatim (mã hóa địa lý dự phòng), SerpAPI (truy vấn khách sạn).

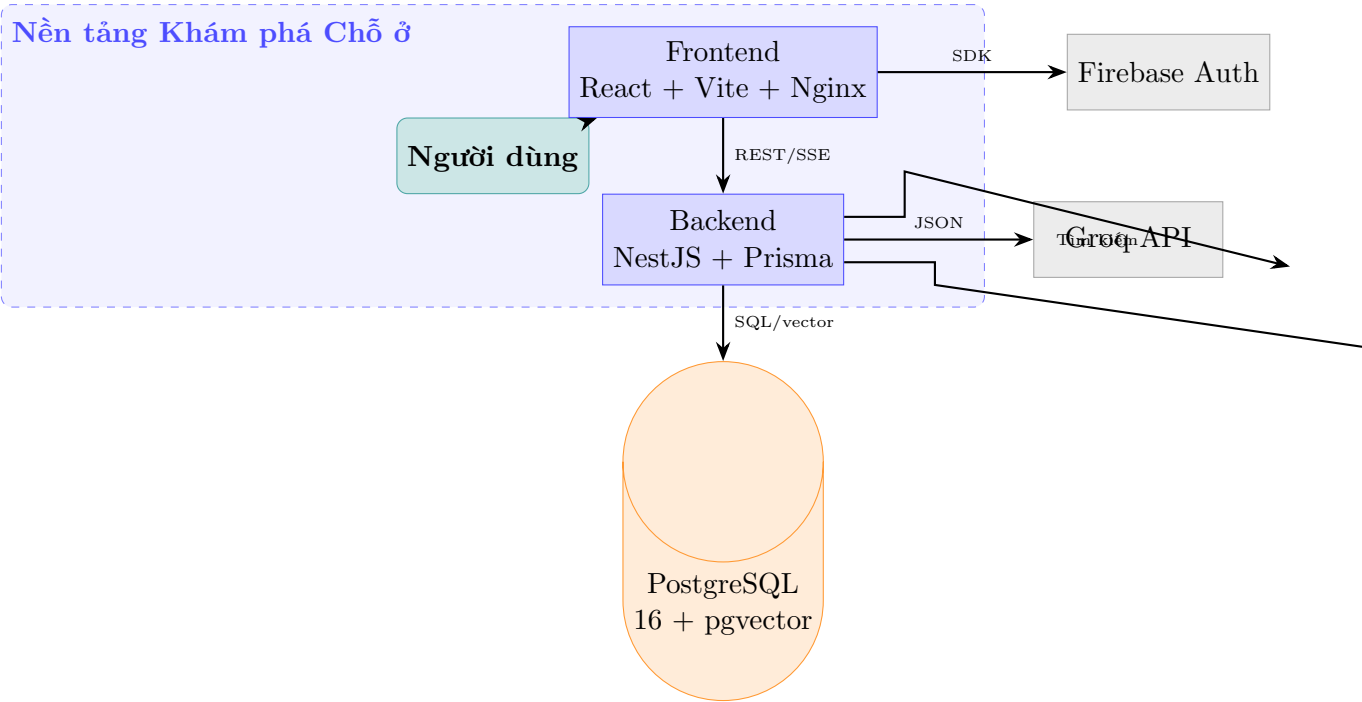
7.3. Mô hình C4

Mô hình C4 được sử dụng để biểu diễn kiến trúc hệ thống ở bốn cấp độ, mỗi cấp độ phục vụ một đối tượng người xem khác nhau.

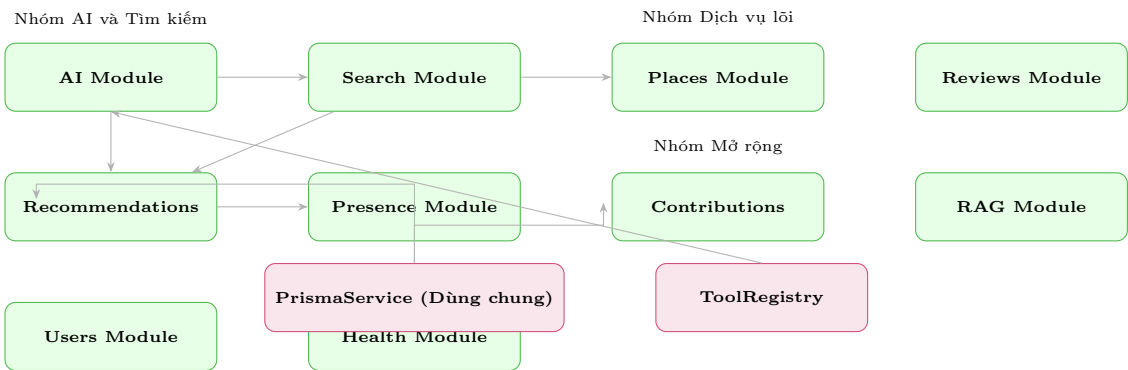
7.3.1 Cấp 1 — System Context



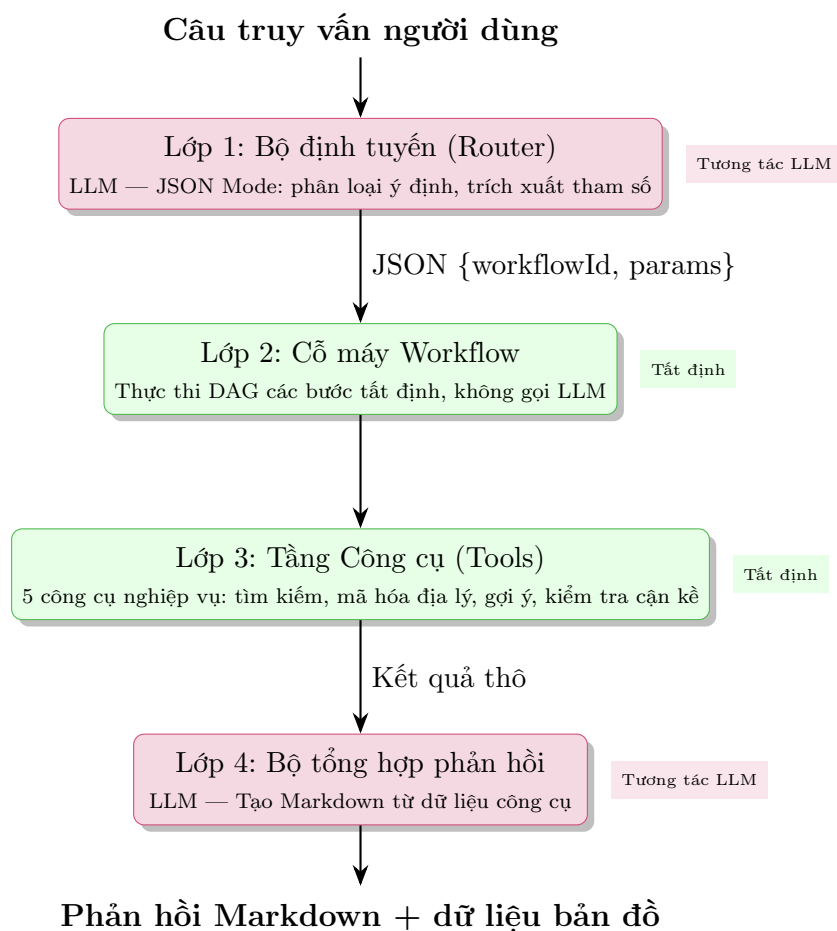
7.3.2 Cấp 2 — Container



7.3.3 Cấp 3 — Component



7.3.4 Cấp 4 — Code (Chi tiết Module AI)

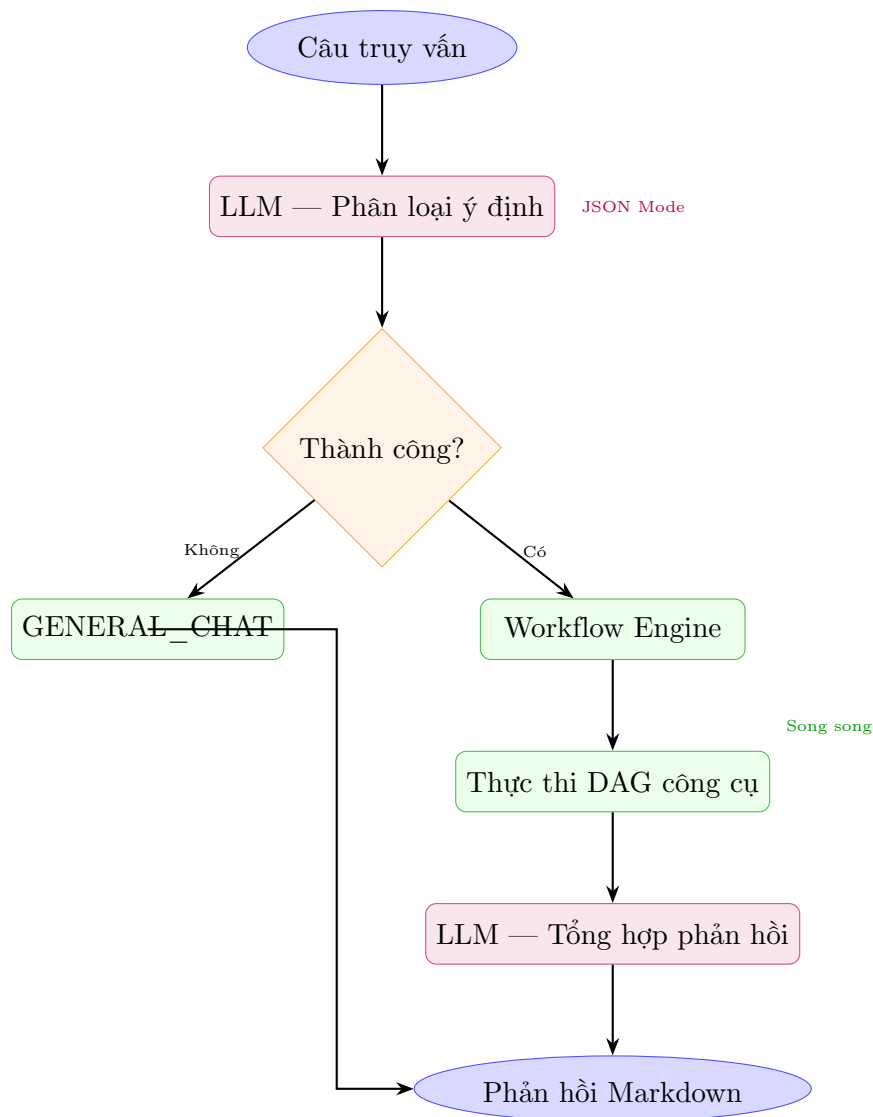


7.4. Thiết kế Giải thuật theo Chức năng

7.4.1 Điều phối AI

Mô tả: Giải thuật điều phối AI chuyển đổi câu truy vấn người dùng thành phản hồi có cấu trúc thông qua kiến trúc bốn lớp.

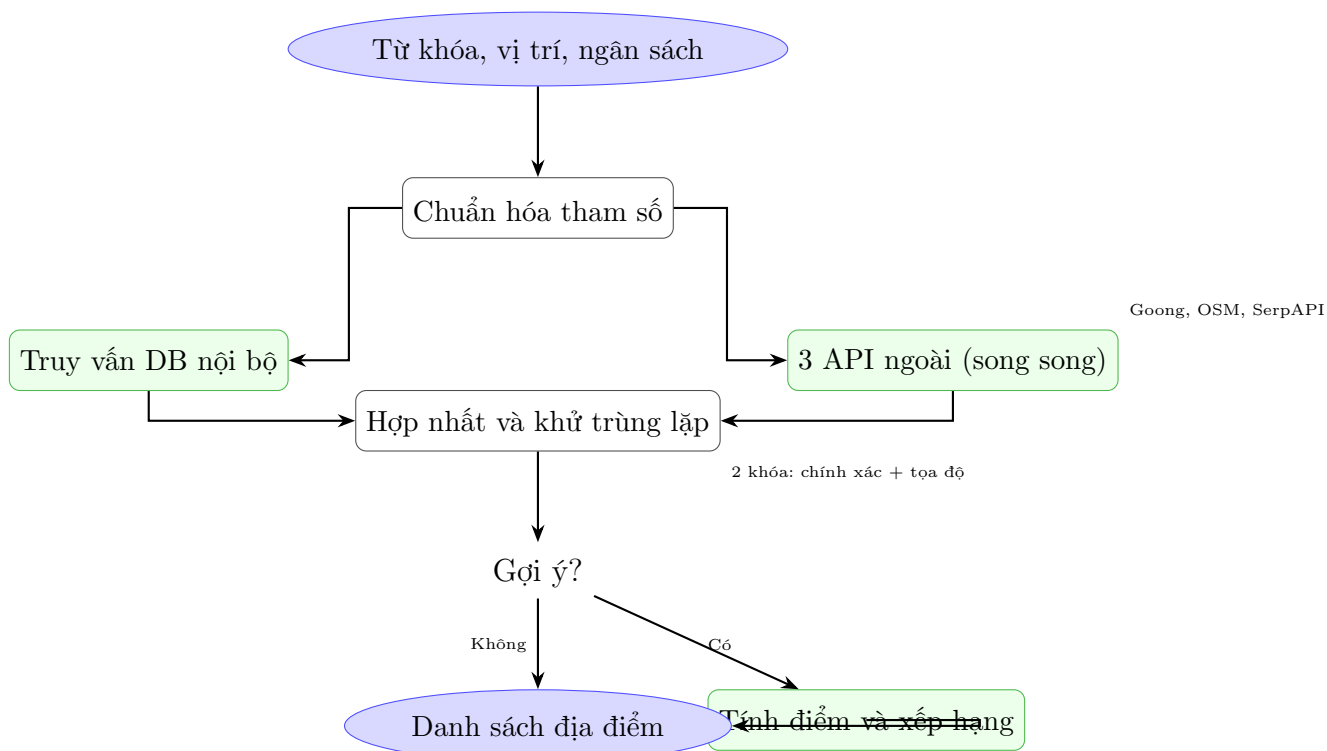
Đầu vào: Câu truy vấn tiếng Việt, lịch sử hội thoại. **Đầu ra:** Phản hồi Markdown kèm dữ liệu bản đồ (searchAction).



Đánh giá: Độ phức tạp $O(|DAG| + |LLM|)$ với $|DAG| \leq 3$. Hai trong bốn lớp không tương tác LLM, đảm bảo tính tất định cho logic nghiệp vụ. Cơ chế an toàn: Router thất bại $\rightarrow$ GENERAL_CHAT; Tool lỗi $\rightarrow$ đánh dấu và tiếp tục.

7.4.2 Tìm kiếm Lai (Hybrid Search)

Mô tả: Kết hợp dữ liệu từ cơ sở dữ liệu nội bộ và ba nhà cung cấp ngoài (Goong, OSM, SerpAPI) để tạo danh sách địa điểm toàn diện.



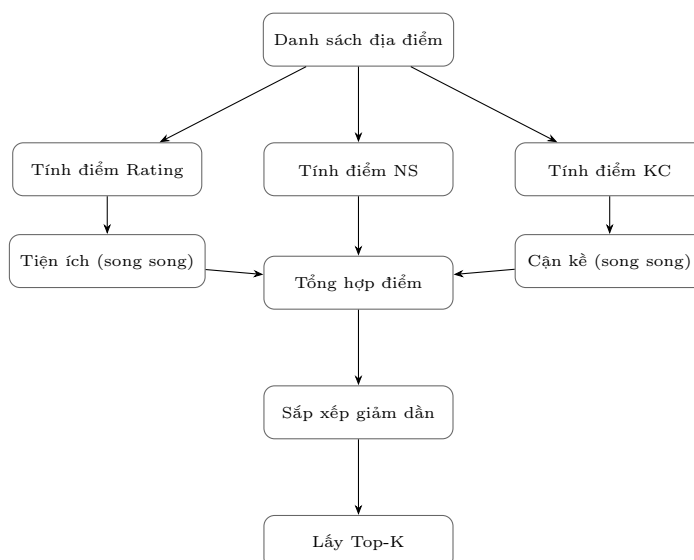
Đánh giá: Độ phức tạp $O(n + m)$. Cách ly lỗi qua catch riêng biệt — một nhà cung cấp thất bại không ảnh hưởng đến các nhà cung cấp khác. Chuỗi dự phòng: Goong → OSM → mảng rỗng.

7.4.3 Gợi ý Xếp hạng

Mô tả: Chuyển đổi kết quả tìm kiếm thô thành danh sách xếp hạng sử dụng mô hình đa yếu tố.

Công thức tính điểm:

$$\text{Điểm} = 0,30 \times \text{Rating} + 0,20 \times \text{Ngân_sách} + 0,20 \times \text{Khoảng_cách} + 0,15 \times \text{Tiện_ích} + 0,15 \times \text{Cận_kề}$$



Khi không có vị trí neo, trọng số khoảng cách và cận kề được phân phối lại cho các yếu tố còn lại. Địa điểm có người đang hiện diện nhận điểm thưởng.

7.4.4 Đường ống RAG

Pipeline phục vụ tìm kiếm ngữ nghĩa trên dữ liệu đóng góp (đánh giá, tệp tin):

- **Tiếp nhận:** Nguồn (review/file) → Phân đoạn văn bản → Sinh embedding → Lưu vào bảng **chunks** với vector.
- **Truy xuất:** Truy vấn → Sinh embedding → Tìm kiếm tương đồng cosine (pgvector) → Xây dựng ngữ cảnh → LLM.

Trạng thái hiện tại: Lược đồ và lưu trữ chunk đã hoàn thiện; sinh embedding đang chờ triển khai.

7.4.5 Các Giải thuật Hỗ trợ

- **Theo dõi hiện diện (Presence):** Check-in/check-out với 3 endpoint. Dữ liệu hiện diện phục vụ UI thời gian thực và làm yếu tố tăng điểm trong gợi ý.
- **Bỏ phiếu Q&A:** Mô hình Reddit với ràng buộc UNIQUE(answer_id, user_id). Giá trị vote (1/-1) được vật chất hóa trên bảng answers.
- **Khử trùng lặp:** Chiến lược hai khóa (chính xác: source:source_place_id; mờ: tên_chuẩn_hóa:lat(3):lng(3)) đảm bảo toàn vẹn khi hợp nhất đa nguồn.

7.5. Đánh giá Hiệu năng và Khả năng Mở rộng

Nút thắt	Mức rủi ro	Biện pháp giảm thiểu
Độ trễ LLM (700ms+)	Cao	Streaming, thực thi công cụ song song trước
Giới hạn tốc độ API ngoài	Trung bình	Chuỗi dự phòng, cache
Tìm kiếm vector tập lớn	Trung bình	Chỉ mục IVFFlat
Truy vấn N+1 Prisma	Thấp	Eager loading

8. Implementation

8.1. Từ Mô hình Thiết kế đến Phân rã Module

Từ các mô hình đã xây dựng ở phần Thiết kế Hệ thống (C4 Model, ERD, sơ đồ tuần tự), nhóm tiến hành phân rã thành các module triển khai theo ba bước:

1. **Phân rã chức năng:** Các khối chức năng trong C4 Container → 10 module NestJS, mỗi module gồm tầng giao tiếp (API), tầng logic (xử lý nghiệp vụ) và tầng dữ liệu (DTO).

2. **Ánh xạ dữ liệu:** Mô hình ERD → 13 lược đồ Prisma với quan hệ, ràng buộc và chỉ mục. Các thao tác vector nhúng (pgvector) được xử lý qua SQL nhờ hạn chế của ORM.
3. **Ánh xạ quy trình:** Sơ đồ tuần tự → workflow registry dạng JSON và các lớp điều phối AI. Mỗi ca sử dụng tương ứng với một workflow cụ thể trong registry.

Các module được tổ chức thành bốn nhóm:

Nhóm	Module	Nguồn gốc
Dịch vụ lõi	Users, Places, Reviews, Search, Health	Kế thừa từ core-service (NestJS)
AI và Tìm kiếm	AI, Recommendations	Chuyển đổi từ FastAPI sang NestJS
Mở rộng MVP	RAG, Presence, Contributions	Phát triển mới
Dùng chung	PrismaService, ToolRegistry, Global Filters	Toàn cục, dùng chung qua DI

8.2. Di trú Kiến trúc: Microservices → Modular Monolith

Hệ thống nguyên bản vận hành với bốn dịch vụ riêng rẽ (Nginx Gateway, Core-service, AI-service, Recommendation-service). Kiến trúc này bộc lộ các vấn đề:

- **Chi phí mạng:** Mỗi yêu cầu chat trải qua ba chặng HTTP, gây độ trễ tích lũy.
- **Trùng lặp logic:** Xác thực, xử lý lỗi, cấu hình bị sao chép giữa các dịch vụ.
- **Phân mảnh kiểu:** Python ↔ TypeScript yêu cầu tuần tự hóa thủ công, dễ sai sót.
- **Phức tạp vận hành:** Bốn Docker image, bốn pipeline triển khai riêng biệt.

Giải pháp: Hợp nhất thành một monolith dạng module NestJS duy nhất. Các lời gọi HTTP liên dịch vụ được thay thế bằng phương thức gọi trực tiếp qua dependency injection, loại bỏ Gateway Nginx trung gian.

8.3. Các Thách thức và Giải pháp

8.3.1 Tính phi tất định của LLM

Thách thức: LLM sinh phản hồi không nhất quán về định dạng, dễ hallucination dữ liệu, và độ trễ khó dự đoán.

Giải pháp — Kiến trúc Bốn Lớp:

- **Lớp 1 (Router):** LLM với JSON Mode → phân loại ý định, trích xuất tham số có cấu trúc. Thất bại → mặc định GENERAL_CHAT.
- **Lớp 2 (Workflow Engine):** Thực thi DAG các bước tất định, không gọi LLM.
- **Lớp 3 (Tool Layer):** Mỗi công cụ có hợp đồng đầu ra chuẩn hóa, không ném ngoại lệ.

- **Lớp 4 (Composer):** Neo phản hồi Markdown vào dữ liệu thực tế từ công cụ, giảm hallucination.

Lợi ích: Workflow mới có thể thêm dưới dạng JSON, không cần biên dịch lại.

8.3.2 Streaming SSE qua POST

Thách thức: SSE chuẩn chỉ hỗ trợ GET, trong khi endpoint chat cần gửi JSON payload và token xác thực.

Giải pháp — Mô hình hai pha:

- Sử dụng `fetch()` với `ReadableStream` thay vì `EventSource`, cho phép POST và đính kèm token.
- Pha 1: Gửi dữ liệu cấu trúc (`searchAction`) ngay sau khi công cụ thực thi — cho phép frontend render bản đồ trước.
- Pha 2: Truyền token LLM dưới dạng delta SSE.
- Header `X-Accel-Buffering`: no để vô hiệu hóa bộ đệm Nginx khi triển khai sau reverse proxy.

8.3.3 Tích hợp Đa Nhà cung cấp Bản đồ

Thách thức: Mỗi API bản đồ trả về định dạng khác nhau; lỗi một nhà cung cấp không được ảnh hưởng toàn bộ tìm kiếm.

Giải pháp:

- Chuẩn hóa giao diện đầu ra cho mọi nhà cung cấp.
- Gọi song song qua `Promise.all` với `.catch()` riêng biệt — đảm bảo cách ly lỗi.
- Chuỗi dự phòng: Goong → OSM → mảng rỗng.

8.3.4 Vector Embedding và Prisma

Thách thức: Prisma chưa hỗ trợ nguyên sinh kiểu `vector` của PostgreSQL.

Giải pháp: Khai báo cột `embedding` dưới dạng `Unsupported("vector")`, thao tác qua SQL thô (`$queryRawUnsafe`).

8.3.5 Xác thực Firebase

Thách thức: Tích hợp Firebase JWT vào vòng đời NestJS guard mà không sao chép logic xác thực trên mọi controller.

Giải pháp: Triển khai guard với `CanActivate`: trích xuất Authorization header → xác thực token qua Firebase Admin SDK → gắn decoded token vào request. Frontend Axios interceptor tự động đính kèm token.

8.4. Các Thách thức Khác

Xử lý đồng thời: Ba mẫu thiết kế được áp dụng: (1) `Promise.all` với catch riêng biệt; (2) ghi vào khóa khác nhau trong shared state để tránh race condition; (3) Active Flag pattern để tránh cập nhật state sau khi component unmount.

Triển khai Docker: Ba container (frontend/Nginx, backend/Node 22, PostgreSQL/pgvector) với chiến lược build đa tầng tận dụng layer cache. Docker override cho môi trường phát triển sử dụng volume ẩn danh tránh xung đột `node_modules`.

Quản lý bản đồ frontend: Chuyển đổi layer bằng opacity, MarkerClusterGroup với chunkedLoading cho hiệu năng, CustomEvent thay vì prop drilling cho tích hợp AI-Search.

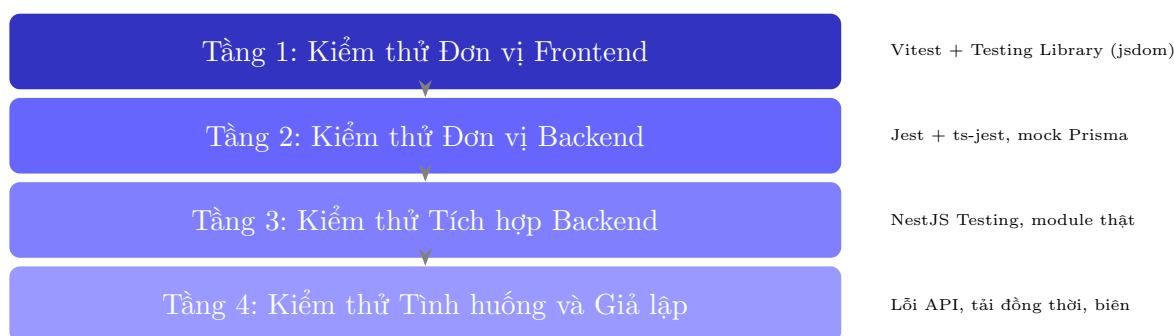
8.5. Kết luận

Quá trình triển khai cho thấy việc chuyển đổi từ mô hình thiết kế sang mã nguồn đòi hỏi liên tục giải quyết các vấn đề thực tế: tính phi tất định của LLM, hạn chế của ORM với kiểu dữ liệu đặc thù, và đảm bảo tính nhất quán xuyên suốt các tầng. Mười hai vấn đề đã được ghi nhận với các mức độ ưu tiên khác nhau.

9. Testing

9.1. Chiến lược Kiểm thử

Nhóm áp dụng chiến lược kiểm thử đa tầng, bao phủ từ đơn vị đến tích hợp, với mục tiêu đảm bảo chất lượng cho toàn bộ hệ thống:



Nguyên tắc: Mock toàn bộ tầng dữ liệu và dịch vụ ngoài ở unit test; kiểm thử tích hợp sử dụng module thật với dependency injection đầy đủ. Mỗi tầng kiểm thử bổ sung và mở rộng cho tầng trước.

9.2. Phạm vi Kiểm thử

9.2.1 Frontend — Giao diện và Tương tác

- **Dịch vụ (Service layer):** Kiểm thử 12 dịch vụ frontend bao gồm API chat, tìm kiếm, địa điểm, đánh giá, hiện diện, đóng góp tệp, gợi ý, RAG, người dùng — xác nhận mỗi service gọi đúng endpoint, xử lý đúng dữ liệu trả về và bắt lỗi HTTP.

- **Context:** Kiểm thử 3 context (Auth, TravelData, Theme) — xác nhận provider render đúng, cung cấp giá trị mặc định và cập nhật state khi có thay đổi.
- **Hooks:** Kiểm thử hook streaming AI — ghép chunk delta SSE, xây dựng prompt ngữ cảnh, xử lý lỗi luồng. Kiểm thử ánh xạ lỗi Firebase — dịch 4 mã lỗi (permission-denied, unauthenticated, network-request-failed, unknown) sang thông báo tiếng Việt.
- **Component:** Kiểm thử 25 component giao diện chính: MapComponent (hiển thị marker, cluster), ChatPanel (gửi/nhận tin nhắn, streaming), PlaceCard (hiển thị thông tin, nút tương tác), SearchBar (tìm kiếm, gợi ý), các component wizard-inputs (LocationPicker, BudgetSelector, TypeSelector) và chat subcomponents (MessageList, StreamingText).
- **Trang:** Kiểm thử 5 trang: HomePage, SearchPage, PlaceDetailPage, ProfilePage, AdminPage — xác nhận layout, điều hướng và tích hợp component.

9.2.2 Backend — Xử lý Nghiệp vụ và Tích hợp

- **Dịch vụ (Services):** Kiểm thử 17 service backend bao phủ toàn bộ module: Người dùng (CRUD, upsert, tìm kiếm), Địa điểm (CRUD, lọc, fuzzy match), Đánh giá (CRUD, điểm số), Tìm kiếm (đa nhà cung cấp, hợp nhất, khử trùng lặp), Gợi ý (tính điểm, xếp hạng, trọng số), Hiện diện (join/leave/get, tải đồng thời), Đóng góp (tạo/lấy/cập nhật tệp), RAG (lưu/truy xuất chunk, xây dựng ngữ cảnh), Hội thoại AI (danh sách/tạo/xóa/tin nhắn), NLP (trích xuất bộ lọc từ văn bản tiếng Việt).
- **Controller:** Kiểm thử 12 controller — xác nhận route đăng ký đúng, validation pipe hoạt động, response format chuẩn.
- **Công cụ AI (Tools):** Kiểm thử 10 tool — SearchPlaces (tìm kiếm lại), RecommendPlaces (xếp hạng), GeocodeAnchor (mã hóa địa lý), ProximityChecker (khoảng cách), NearbyAmenities (tiện ích lân cận), và các tool hỗ trợ. Mỗi tool được kiểm tra cả luồng thành công và thất bại.
- **Guard/Filter/Interceptor:** Kiểm thử FirebaseAuthGuard (thiếu header, sai định dạng, token không hợp lệ), AllExceptionsFilter (HttpException, lỗi không xác định), LoggingInterceptor (định nghĩa, phương thức).

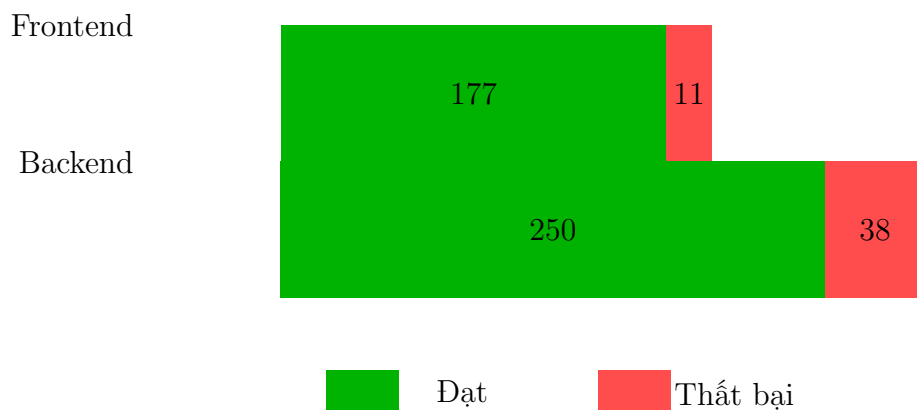
9.2.3 Các Tình huống và Giả lập Đặc thù

- **Giả lập lỗi nhà cung cấp ngoài:** Khi Goong API thất bại, hệ thống tự động chuyển sang OSM; khi cả hai thất bại, trả về kết quả rỗng an toàn. Một nhà cung cấp lỗi không ảnh hưởng đến kết quả từ các nhà cung cấp khác.
- **Xử lý workflow thất bại:** Router phân tích lỗi → dự phòng GENERAL_CHAT; tool lỗi → đánh dấu lỗi và tiếp tục thực thi workflow.
- **SSE streaming đầu cuối:** Xác minh kết nối, nhận chunk delta, xử lý finish, buffer ghép mảnh.
- **Dữ liệu rỗng và trường hợp biên:** Tìm kiếm không kết quả, hội thoại mới, người dùng chưa có profile, địa điểm không có đánh giá.

- **Tải đồng thời:** 100 người dùng join cùng lúc vào cùng địa điểm — kiểm tra tính toàn vẹn của cấu trúc dữ liệu.

9.3. Kết quả Kiểm thử

Kết quả kiểm thử tổng thể được thống kê như sau:



Frontend — 177/188 đạt (94,1%)

12 dịch vụ, 3 context, 3 hooks, 5 trang, 25 component

Backend — 250/288 đạt (86,8%)

17 services, 12 controllers, 10 tools, 3 guard/filter/interceptor

Tổng thể — 427/476 đạt (89,7%)

Phân bố kiểm thử theo nhóm chức năng backend:



Phân bố kiểm thử theo nhóm chức năng frontend:

Nhóm chức năng	Số dịch vụ/Component	Kết
Dịch vụ API (aiService, searchService, placeService, ...)	12 services	11/11
Ngữ cảnh (AuthContext, TravelDataContext, ThemeContext)	3 contexts	2/3
Hooks (streamingChat, firestoreError)	3 hooks	3/3
Component giao diện (Map, Chat, PlaceCard, SearchBar, ...)	25 components	25/25
Trang (Home, Search, PlaceDetail, Profile, Admin)	5 pages	5/5

Ghi chú: 11 ca thất bại frontend và 38 ca thất bại backend đều liên quan đến mock Firebase/Firestore chưa hoàn chỉnh — không phải lỗi logic nghiệp vụ. Tất cả module chức năng chính đều được kiểm thử thành công.

9.4. Cải tiến Sau Kiểm thử

Quá trình kiểm thử đã phát hiện và dẫn đến các cải tiến sau:

- **Cải tiến xử lý lỗi:** Chuẩn hóa phản hồi lỗi thành cấu trúc JSON thống nhất với timestamp, loại bỏ try/catch trùng lặp ở controller.
- **Cải tiến khử trùng lặp:** Bổ sung khóa mờ theo tọa độ (lat:lng độ chính xác 3 chữ số thập phân) bên cạnh khóa chính xác, giúp phát hiện trùng lặp khi ID nhà cung cấp khác nhau.
- **Cải tiến buffer SSE:** Cả backend và frontend được đồng bộ buffer pattern để xử lý chunk SSE bị phân mảnh do TCP.
- **Cải tiến Presence:** Thêm kiểm tra tải 100 người dùng đồng thời, đảm bảo cấu trúc dữ liệu hoạt động ổn định dưới tải cao.
- **Cải tiến NLP:** Tham số hóa 14 văn bản tiếng Việt thực tế, bao phủ đa dạng biến thể ngôn ngữ (tên tỉnh thành, loại hình, mức giá).

9.5. Hướng Phát triển

Các khoảng trống kiểm thử đã được xác định:

- Kiểm thử tích hợp với PostgreSQL + pgvector thực (hiện đang mock Prisma).
- Pipeline CI/CD tự động hóa quy trình kiểm thử.
- Kiểm thử hiệu năng với tập dữ liệu lớn (10^5 + địa điểm).
- Kiểm thử người dùng (usability testing) có tổ chức.

10. Demo

11. Deployment

12. Logbook: Plan and Actual Workload

13. AI usage and declaration

14. Kết luận và hướng phát triển

14.1. Kết luận

Trong quá trình thực hiện đề tài, nhóm đã xây dựng thành công ứng dụng web toàn diện với kiến trúc hiện đại, tích hợp các công nghệ AI tiên tiến. Hệ thống được phát triển trên nền tảng React (frontend) và NestJS (backend), kết hợp với Prisma ORM, Firebase Authentication, và các dịch vụ AI đa nền tảng (Gemini, Cloudflare AI, FreeModel).

Các chức năng đã được triển khai thành công bao gồm:

- Chức năng tìm kiếm qua chatbot
- Chức năng so sánh các địa điểm lưu trú
- Chức năng trích xuất thông tin chi tiết và đánh giá của AI về một địa điểm lưu trú cụ thể

14.2. Những kiến thức và kỹ năng đạt được

Môn học và đồ án lần này đã mang lại cho chúng em những bài học vô cùng thực tế, giúp thu hẹp khoảng cách giữa lý thuyết và ứng dụng:

- **Vận dụng 4 trụ cột cốt lõi của Tư duy Máy tính (Computational Thinking):**
Thông qua đồ án, chúng em đã làm chủ và áp dụng thành công mô hình tư duy này để giải quyết bài toán thực tế. Cụ thể, chúng em thực hiện
 1. *Phân rã bài toán* (Decomposition) để chia nhỏ hệ thống thành các module độc lập (*AiModule* và *RecommendationsModule*);
 2. *Nhận biết mẫu* (Pattern Recognition) để tìm ra khuôn mẫu trong hành vi tìm kiếm của khách hàng;
 3. *Trừu tượng hóa* (Abstraction) để lọc bỏ thông tin nhiễu, trích xuất tham số cốt lõi qua *Intent Parsing*;
 4. *Thiết kế thuật toán* (Algorithm Design) để xây dựng luồng vận hành (Workflow) tối ưu cho Chatbot.

Các kỹ năng này giúp chúng em định hình được tư duy logic vững chắc cho các dự án phức tạp sau này.

- **Tư duy khai thác Trí tuệ nhân tạo:** Chúng em đã học được cách viết cấu trúc câu lệnh (Prompt Engineering) và cách ứng dụng AI một cách hiệu quả, biến AI thành một công cụ đồng hành đắc lực để tối ưu hóa việc tìm kiếm thông tin thay vì lạm dụng một cách máy móc.
- **Làm quen với các công nghệ mới:** Đồ án là cơ hội để chúng em tiếp cận và biết cách sử dụng hàng loạt công cụ, ngôn ngữ và nền tảng cốt lõi trong phát triển phần mềm hiện đại như HTML, JavaScript, nền tảng cơ sở dữ liệu Firebase, bản đồ OpenStreetMap,... Dù thời gian có hạn và chưa thể tìm hiểu quá sâu sắc từng công nghệ, nhưng đây là nền móng cực kỳ quan trọng cho các môn học chuyên ngành tiếp theo.
- **Kỹ năng làm việc nhóm và Quản lý mã nguồn:** Biết cách sử dụng GitHub để quản lý các phiên bản code, phối hợp phân chia công việc trong nhóm một cách hệ thống.
- **Tư duy Hệ thống:** Hiểu rõ bức tranh toàn cảnh về cách một hệ thống phần mềm vận hành trong thực tế (cách Frontend tương tác với Backend, cách gọi API và truy vấn dữ liệu từ Database).

14.3. Hướng phát triển trong tương lai

Nhằm nâng cấp và hoàn thiện dự án thành một sản phẩm có tính ứng dụng thực tế cao hơn, các mục tiêu trong tương lai bao gồm:

- **Nghiên cứu sâu về RAG và LLM:** Tối ưu hóa sâu hơn kỹ thuật Retrieval-Augmented Generation (RAG) kết hợp với các mô hình ngôn ngữ lớn (như Groq) để nâng cao độ chính xác, giảm thiểu hiện tượng "ảo tưởng" (hallucination) của Chatbot khi trả lời khách hàng.
- **Tối ưu hóa UI/UX:** Thiết kế lại giao diện người dùng thân thiện, trực quan hơn trên cả hai nền tảng Web và Mobile, tối ưu hóa các bộ lọc nâng cao để nâng cao trải nghiệm của các nhóm đối tượng Target Users.